

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO: ỨNG DỤNG PHÂN TÁN
*Đề tài: Nghiên cứu và mở rộng hệ thống truyền thông
điện phân tán NSQ*

Giảng viên hướng dẫn: TS. Nguyễn Lệ Thu.

Thành viên nhóm:

1. Nguyễn Thị Thùy Linh – 23010633.
2. Nguyễn Anh Quân – 23010375.

Lớp: CSE702063-1-3-25(N03).

Hà Nội, 2026.

MỤC LỤC

1. DỰ ÁN ĐÃ CHỌN	1
1.1 Giới thiệu dự án	1
1.2 Kiến trúc hệ thống	1
1.3 Đặc điểm của hệ thống phân tán NSQ.....	2
2. MỤC ĐÍCH, CHỨC NĂNG VÀ ỨNG DỤNG THỰC TẾ	4
2.1 Mục đích của dự án	4
2.2 Chức năng chính của hệ thống	4
2.3 Ứng dụng thực tế	6
2.3.1. Hệ thống thương mại điện tử.....	6
2.3.2. Hệ thống Microservices.....	6
2.3.3. Hệ thống xử lý Log.....	6
2.3.4. Hệ thống Internet of Things (IoT)	6
2.3.5. Hệ thống xử lý dữ liệu thời gian thực	7
2.4 Đánh giá.....	7
3. THIẾT LẬP CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM	8
3.1 Môi trường thực nghiệm.....	8
3.2 Tải và biên dịch mã nguồn	8
3.3 Khởi động hệ thống NSQ	9
3.4 Thực nghiệm cân bằng tải (Load Balancing)	9
3.5 Thực nghiệm khả năng chịu lỗi (Fault Tolerance)	10
3.6 Đánh giá kết quả thực nghiệm.....	11
4. PHÁT TRIỂN TÍNH NĂNG MỚI	12
4.1. Consumer Load Balancing Monitor	12
4.1.1. Mục đích	12
4.1.2. Thiết kế	12
4.1.3. Cài đặt.....	13
4.1.4. Kết quả.....	13
4.1.5. Liên hệ với hệ phân tán	14

4.1.6. Đánh giá.....	14
4.2. Distributed Node Health Monitoring.....	15
4.2.1. Mục đích.....	15
4.2.2. Thiết kế.....	15
4.2.3. Cài đặt.....	15
4.2.4. Kết quả.....	16
4.2.5. Kiểm thử.....	16
4.2.6. Liên hệ với hệ phân tán	17
4.2.7. Đánh giá.....	17
4.3 Tổng kết các tính năng mở rộng.....	18
5. ĐÓNG GÓP CỦA THÀNH VIÊN	20
5.1 Phân công công việc.....	20
5.2 Kết quả làm việc nhóm.....	20
KẾT LUẬN	22

1. DỰ ÁN ĐÃ CHỌN

1.1 Giới thiệu dự án

Trong các hệ thống phần mềm hiện đại, việc trao đổi dữ liệu giữa các thành phần của hệ thống đóng vai trò rất quan trọng. Khi số lượng người dùng và khối lượng dữ liệu ngày càng tăng, các hệ thống cần một cơ chế truyền thông điệp hiệu quả nhằm đảm bảo khả năng mở rộng, tính sẵn sàng và khả năng chịu lỗi. Một trong những giải pháp được sử dụng phổ biến hiện nay là hệ thống hàng đợi thông điệp phân tán (Distributed Messaging System).

Sau khi tìm hiểu các dự án trong kho lưu trữ Awesome Distributed System Projects, nhóm lựa chọn nghiên cứu dự án NSQ. Đây là một nền tảng truyền thông điệp phân tán mã nguồn mở được phát triển bởi Bitly và được viết hoàn toàn bằng ngôn ngữ Golang. NSQ được thiết kế nhằm xử lý lượng lớn thông điệp theo thời gian thực, hỗ trợ kiến trúc phân tán và có khả năng mở rộng theo chiều ngang.

Thông tin dự án:

- Tên dự án: NSQ
- Địa chỉ mã nguồn: <https://github.com/nsqio/nsq> .
- Nguồn dự án: Dự án được lựa chọn từ kho lưu trữ Awesome Distributed System Projects theo yêu cầu của học phần Ứng dụng phân tán.
- Ngôn ngữ phát triển: Golang
- Loại hệ thống: Distributed Messaging Platform
- Giấy phép: MIT License

NSQ được sử dụng để kết nối các dịch vụ trong hệ thống phân tán thông qua mô hình Publish/Subscribe. Hệ thống cho phép Producer gửi thông điệp đến các Topic, sau đó các Consumer sẽ nhận và xử lý các thông điệp này một cách độc lập. Điều này giúp giảm sự phụ thuộc giữa các thành phần trong hệ thống và tăng khả năng mở rộng.

1.2 Kiến trúc hệ thống

NSQ được xây dựng theo mô hình truyền thông điệp phân tán (Distributed Messaging System), cho phép các thành phần trong hệ thống giao tiếp với nhau thông qua cơ chế Publish/Subscribe. Kiến trúc của NSQ được thiết kế đơn giản nhưng vẫn đảm bảo khả năng mở rộng, hiệu năng cao và khả năng chịu lỗi trong môi trường phân tán.

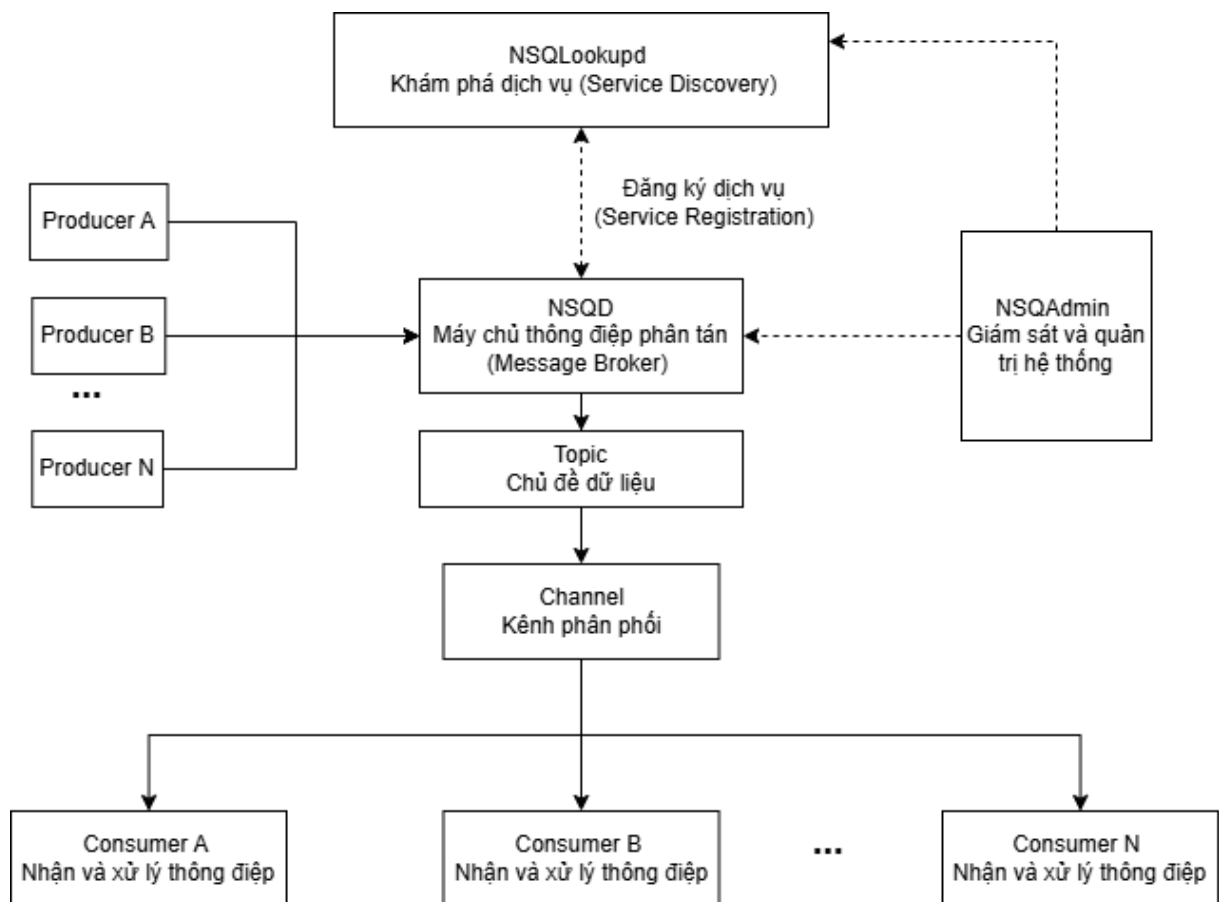
Hệ thống gồm ba thành phần chính:

NSQLookupd: NSQLookupd là thành phần thực hiện chức năng Service Discovery. Thành phần này lưu trữ thông tin về các NSQD đang hoạt động trong hệ thống và hỗ trợ các Consumer tìm kiếm NSQD để kết nối.

NSQD: NSQD là thành phần cốt lõi của hệ thống, chịu trách nhiệm tiếp nhận, lưu trữ và phân phối thông điệp. Đây là nơi các Producer gửi dữ liệu và các Consumer nhận dữ liệu để xử lý.

NSQAdmin: NSQAdmin là giao diện web hỗ trợ giám sát và quản trị hệ thống. Thông qua NSQAdmin, người quản trị có thể theo dõi Topic, Channel, Consumer, lưu lượng thông điệp và các thông số hoạt động của hệ thống theo thời gian thực.

Trong mô hình hoạt động của NSQ, Producer gửi thông điệp đến NSQD. Các thông điệp được lưu trong các Topic và phân phối đến các Consumer thông qua các Channel. NSQLookupd đóng vai trò trung gian giúp các thành phần trong hệ thống tự động tìm thấy nhau mà không cần cấu hình thủ công.



Hình 1. Kiến trúc tổng quan của hệ thống NSQ.

1.3 Đặc điểm của hệ thống phân tán NSQ

NSQ sở hữu nhiều đặc điểm tiêu biểu của một hệ thống phân tán hiện đại:

- Khả năng mở rộng (Scalability): Hệ thống cho phép bổ sung thêm nhiều Producer hoặc Consumer mà không cần thay đổi kiến trúc hiện tại.
- Cân bằng tải (Load Balancing): Thông điệp được phân phối tự động giữa các Consumer trong cùng một Channel nhằm tối ưu tài nguyên xử lý.

- Khả năng chịu lỗi (Fault Tolerance): Khi một Consumer gặp sự cố hoặc ngừng hoạt động, các Consumer còn lại vẫn có thể tiếp tục xử lý thông điệp mà không làm gián đoạn hệ thống.
- Xử lý bất đồng bộ (Asynchronous Processing): Producer và Consumer hoạt động độc lập với nhau, giúp giảm sự phụ thuộc giữa các dịch vụ trong hệ thống.
- Hiệu năng cao: NSQ được phát triển bằng Golang nên có khả năng xử lý lượng lớn thông điệp với độ trễ thấp và hiệu năng cao.
- Khả năng phân tán (Distributed Architecture): Hệ thống có thể triển khai trên nhiều máy chủ khác nhau, các node giao tiếp với nhau thông qua mạng.

Nhờ các đặc điểm trên, NSQ được sử dụng rộng rãi trong các hệ thống Microservices, thương mại điện tử, xử lý log, hệ thống IoT và các ứng dụng yêu cầu xử lý dữ liệu thời gian thực.

Trong phạm vi đồ án, nhóm tập trung thực nghiệm hai đặc tính quan trọng của hệ thống phân tán là khả năng cân bằng tải (Load Balancing) và khả năng chịu lỗi (Fault Tolerance). Đồng thời nhóm phát triển thêm hai chức năng mới trên giao diện NSQAdmin nhằm hỗ trợ giám sát hệ thống gồm Priority Topic Monitoring và Consumer Health Monitoring.

2. MỤC ĐÍCH, CHỨC NĂNG VÀ ỨNG DỤNG THỰC TẾ

2.1 Mục đích của dự án

Trong các hệ thống phần mềm hiện đại, đặc biệt là các hệ thống có quy mô lớn hoặc được xây dựng theo kiến trúc Microservices, việc trao đổi dữ liệu giữa các thành phần là một yêu cầu quan trọng. Nếu các dịch vụ giao tiếp trực tiếp với nhau, hệ thống sẽ dễ gặp các vấn đề như quá tải, phụ thuộc chặt chẽ giữa các thành phần và khó mở rộng.

NSQ được xây dựng nhằm giải quyết các vấn đề trên thông qua cơ chế truyền thông điệp phân tán. Hệ thống đóng vai trò là cầu nối trung gian giữa các dịch vụ, cho phép các thành phần trao đổi dữ liệu một cách bất đồng bộ mà không cần kết nối trực tiếp với nhau.

Mục tiêu chính của NSQ là cung cấp một nền tảng truyền thông điệp phân tán giúp các dịch vụ giao tiếp theo cơ chế bất đồng bộ, giảm sự phụ thuộc giữa các thành phần và hỗ trợ triển khai các hệ thống có khả năng mở rộng cao. Bên cạnh đó, NSQ còn giúp nâng cao khả năng chịu lỗi và tối ưu hiệu năng xử lý dữ liệu trong môi trường thời gian thực.

Nhờ đó, NSQ giúp giảm tải cho các dịch vụ xử lý nghiệp vụ và nâng cao tính ổn định của toàn bộ hệ thống.

2.2 Chức năng chính của hệ thống

NSQ hoạt động dựa trên mô hình Publish/Subscribe, trong đó các thông điệp được gửi từ Producer đến Consumer thông qua Topic và Channel.

Producer: Producer là thành phần chịu trách nhiệm tạo và gửi thông điệp vào hệ thống.

Các chức năng chính:

- Tạo dữ liệu hoặc sự kiện.
- Gửi thông điệp đến Topic.
- Không cần biết Consumer nào sẽ xử lý thông điệp.

Ví dụ: Trong một website thương mại điện tử, khi khách hàng đặt hàng thành công, hệ thống có thể gửi một thông điệp chứa thông tin đơn hàng đến NSQ.

Topic: Topic là nơi tiếp nhận và lưu trữ các thông điệp từ Producer.

Các chức năng chính:

- Nhận thông điệp từ Producer.
- Quản lý luồng dữ liệu.
- Phân phối thông điệp đến các Channel.

Một Topic có thể phục vụ nhiều mục đích khác nhau như:

- order-topic
- payment-topic

- notification-topic

Channel: Channel là thành phần trung gian giữa Topic và Consumer.

Các chức năng chính:

- Nhận thông điệp từ Topic.
- Phân phối thông điệp đến Consumer.
- Thực hiện cân bằng tải giữa các Consumer.

Mỗi Channel hoạt động độc lập và đảm bảo rằng mỗi thông điệp chỉ được xử lý một lần trong cùng một Channel.

Consumer: Consumer là thành phần nhận và xử lý thông điệp.

Các chức năng chính:

- Kết nối đến Channel.
- Nhận thông điệp từ NSQ.
- Thực hiện xử lý nghiệp vụ.
- Xác nhận hoàn thành xử lý thông điệp với NSQ.

Ví dụ: Khi nhận được thông tin đơn hàng mới, Consumer có thể:

- Gửi email xác nhận.
- Cập nhật kho hàng.
- Tạo hóa đơn.

NSQLookupd: NSQLookupd thực hiện chức năng Service Discovery trong hệ thống.

Các chức năng chính:

- Quản lý thông tin các node NSQD.
- Hỗ trợ Consumer tìm kiếm NSQD.
- Tự động cập nhật trạng thái hệ thống.

Nhờ có NSQLookupd, các Consumer không cần biết chính xác địa chỉ của từng NSQD.

NSQAdmin: NSQAdmin là giao diện quản trị của hệ thống.

Các chức năng chính:

- Theo dõi Topic.
- Theo dõi Channel.
- Theo dõi Consumer.
- Hiện thị số lượng thông điệp.
- Giám sát hoạt động hệ thống.

Trong quá trình thực nghiệm, nhóm sử dụng NSQAdmin để quan sát hoạt động của Producer, Consumer và các thông điệp được truyền trong hệ thống. Đồng thời, đây

cũng là nền tảng để nhóm phát triển các tính năng mở rộng phục vụ công tác giám sát và quản trị hệ thống phân tán.

2.3 Ứng dụng thực tế

Hiện nay, các hệ thống truyền thông điệp phân tán được sử dụng rộng rãi trong nhiều lĩnh vực khác nhau.

2.3.1. Hệ thống thương mại điện tử

Khi khách hàng đặt hàng thành công, hệ thống có thể tạo các thông điệp để:

- Xử lý thanh toán.
- Gửi email xác nhận.
- Cập nhật tồn kho.
- Tạo vận đơn giao hàng.

Việc sử dụng hàng đợi thông điệp giúp giảm tải cho hệ thống và tăng tốc độ phản hồi cho người dùng.

2.3.2. Hệ thống Microservices

Trong kiến trúc Microservices, các dịch vụ thường hoạt động độc lập.

NSQ đóng vai trò trung gian giúp:

- Giảm sự phụ thuộc giữa các dịch vụ.
- Tăng khả năng mở rộng.
- Đảm bảo dữ liệu được truyền tải ổn định.

Trong phạm vi đề án, nhóm sử dụng NSQ để mô phỏng quá trình truyền và xử lý thông điệp giữa nhiều Producer và Consumer, qua đó đánh giá khả năng cân bằng tải và khả năng chịu lỗi của hệ thống trong môi trường phân tán.

2.3.3. Hệ thống xử lý Log

Các hệ thống lớn thường sinh ra hàng triệu bản ghi log mỗi ngày.

NSQ có thể được sử dụng để:

- Thu thập log.
- Phân phối log đến các dịch vụ phân tích.
- Giảm tải cho hệ thống ghi log.

2.3.4. Hệ thống Internet of Things (IoT)

Trong các hệ thống IoT, hàng nghìn thiết bị có thể gửi dữ liệu liên tục.

NSQ giúp:

- Thu thập dữ liệu cảm biến.
- Phân phối dữ liệu đến các dịch vụ xử lý.
- Đảm bảo khả năng mở rộng khi số lượng thiết bị tăng lên.

2.3.5. Hệ thống xử lý dữ liệu thời gian thực

Các hệ thống như: Giám sát giao thông , Theo dõi tài chính, Giám sát an ninh, Hệ thống cảnh báo thời gian thực đều có thể sử dụng NSQ để xử lý dữ liệu với độ trễ thấp và hiệu năng cao.

2.4 Đánh giá

Qua quá trình tìm hiểu và thực nghiệm, nhóm nhận thấy NSQ là một nền tảng truyền thông điệp phân tán mạnh mẽ với kiến trúc đơn giản nhưng hiệu quả. Hệ thống hỗ trợ tốt các đặc tính quan trọng của hệ phân tán như cân bằng tải, khả năng chịu lỗi và khả năng mở rộng. Ngoài các chức năng có sẵn, nhóm còn phát triển thêm hai tính năng gồm Priority Topic Monitoring và Consumer Health Monitoring nhằm hỗ trợ việc giám sát và quản trị hệ thống hiệu quả hơn. Các tính năng này được xây dựng dựa trên việc khai thác thông tin từ nhiều thành phần trong hệ thống NSQ, góp phần thể hiện các đặc trưng của hệ thống phân tán như khả năng mở rộng, giám sát tập trung và quản lý nhiều nút xử lý.

3. THIẾT LẬP CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

3.1 Môi trường thực nghiệm

Nhóm thực hiện cài đặt và thử nghiệm hệ thống NSQ trên nền tảng GitHub Codespaces nhằm tận dụng môi trường Linux trực tuyến, hỗ trợ đầy đủ Golang và các công cụ phát triển cần thiết cho việc xây dựng và vận hành hệ thống phân tán.

Thành phần	Giá trị
Hệ điều hành	Linux (GitHub Codespaces)
Ngôn ngữ lập trình	Golang
IDE	Visual Studio Code
Quản lý mã nguồn	GitHub
Nền tảng thực nghiệm	GitHub Codespaces
Distributed Messaging Platform	NSQ
Trình duyệt	Google Chrome
Công cụ quản trị	NSQAdmin

Bảng 3.1. Môi trường thực nghiệm

3.2 Tải và biên dịch mã nguồn

Nhóm lựa chọn dự án NSQ từ GitHub và tiến hành tải mã nguồn về môi trường Codespaces.

Lệnh tải mã nguồn: `git clone https://github.com/nsqio/nsq.git`

Sau khi tải mã nguồn, nhóm tiến hành biên dịch dự án bằng Makefile được cung cấp sẵn.

Lệnh build: `make`

```
@ttlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ ls -l build
total 94564
-rwxrwxrwx 1 codespace codespace 9413781 Jun 15 14:21 nsq_stat
-rwxrwxrwx 1 codespace codespace 10014244 Jun 15 14:21 nsq_tail
-rwxrwxrwx 1 codespace codespace 10145843 Jun 15 14:21 nsq_to_file
-rwxrwxrwx 1 codespace codespace 10091226 Jun 15 14:21 nsq_to_http
-rwxrwxrwx 1 codespace codespace 10109840 Jun 15 14:21 nsq_to_nsq
-rwxrwxrwx 1 codespace codespace 15308652 Jun 15 14:21 nsqadmin
-rwxrwxrwx 1 codespace codespace 12482057 Jun 15 14:21 nsqd
-rwxrwxrwx 1 codespace codespace 10612256 Jun 15 14:21 nsqlookupd
-rwxrwxrwx 1 codespace codespace 8635754 Jun 15 14:24 to_nsq
```

Hình 2. Các tệp thực thi được tạo thành công sau quá trình biên dịch.

Kết quả cho thấy toàn bộ thành phần của NSQ được biên dịch thành công mà không phát sinh lỗi.

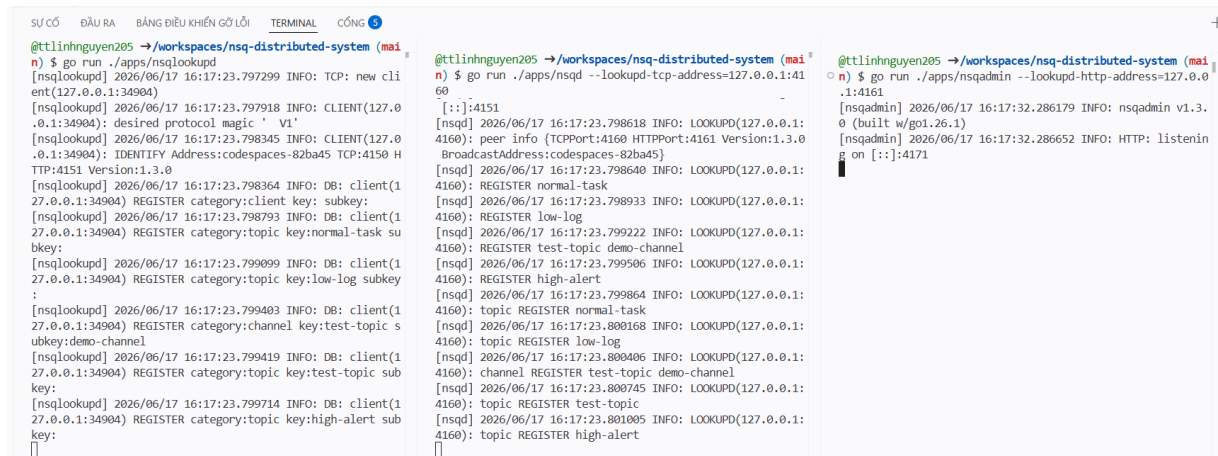
3.3 Khởi động hệ thống NSQ

Sau khi build thành công, nhóm tiến hành khởi động các thành phần của hệ thống theo trình tự sau:

Bước 1: Khởi động NSQLookupd: NSQLookupd thực hiện chức năng Service Discovery, cho phép các thành phần trong hệ thống tự động tìm thấy nhau.

Bước 2: Khởi động NSQD: NSQD là thành phần tiếp nhận, lưu trữ và phân phối thông điệp trong hệ thống.

Bước 3: Khởi động NSQAdmin: NSQAdmin cho phép giám sát Topic, Channel, Consumer và trạng thái hoạt động của hệ thống.



```
@tlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ go run ./apps/nsqllookupd
[nsqllookupd] 2026/06/17 16:17:23.797299 INFO: TCP: new client(127.0.0.1:34904)
[nsqllookupd] 2026/06/17 16:17:23.797918 INFO: CLIENT(127.0.0.1:34904): desired protocol magic ' V1'
[nsqllookupd] 2026/06/17 16:17:23.798345 INFO: CLIENT(127.0.0.1:34904): IDENTIFY Address:codespaces-82ba45 TCP:4150 HTTP:4151 Version:1.3.0
[nsqllookupd] 2026/06/17 16:17:23.798364 INFO: DB: client(127.0.0.1:34904) REGISTER category:client key:subkey:
[nsqllookupd] 2026/06/17 16:17:23.798793 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:normal-task subkey:
[nsqllookupd] 2026/06/17 16:17:23.799099 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:low-log subkey:
[nsqllookupd] 2026/06/17 16:17:23.799483 INFO: DB: client(127.0.0.1:34904) REGISTER category:channel key:test-topic subkey:demo-channel
[nsqllookupd] 2026/06/17 16:17:23.799419 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:test-topic subkey:
[nsqllookupd] 2026/06/17 16:17:23.799714 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:high-alert subkey:

@tlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ go run ./apps/nsqd --lookupd-tcp-address=127.0.0.1:4160
[nsqd] 2026/06/17 16:17:23.798618 INFO: LOOKUPD(127.0.0.1:4160): peer info {TCP:4160 HTTP:4161 Version:1.3.0 BroadcastAddress:codespaces-82ba45}
[nsqd] 2026/06/17 16:17:23.798640 INFO: LOOKUPD(127.0.0.1:4160): REGISTER normal-task
[nsqd] 2026/06/17 16:17:23.798933 INFO: LOOKUPD(127.0.0.1:4160): REGISTER low-log
[nsqd] 2026/06/17 16:17:23.799222 INFO: LOOKUPD(127.0.0.1:4160): REGISTER test-topic demo-channel
[nsqd] 2026/06/17 16:17:23.799506 INFO: LOOKUPD(127.0.0.1:4160): REGISTER high-alert
[nsqd] 2026/06/17 16:17:23.799864 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER normal-task
[nsqd] 2026/06/17 16:17:23.800168 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER low-log
[nsqd] 2026/06/17 16:17:23.800406 INFO: LOOKUPD(127.0.0.1:4160): channel REGISTER test-topic demo-channel
[nsqd] 2026/06/17 16:17:23.800745 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER test-topic
[nsqd] 2026/06/17 16:17:23.801005 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER high-alert

@tlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ go run ./apps/nsqadmin --lookupd-http-address=127.0.0.1:4161
[nsqadmin] 2026/06/17 16:17:32.286179 INFO: nsqadmin v1.3.0 (built w/go1.26.1)
[nsqadmin] 2026/06/17 16:17:32.286652 INFO: HTTP: listening on [::]:4171
```

Hình 3. Các thành phần NSQ hoạt động thành công.

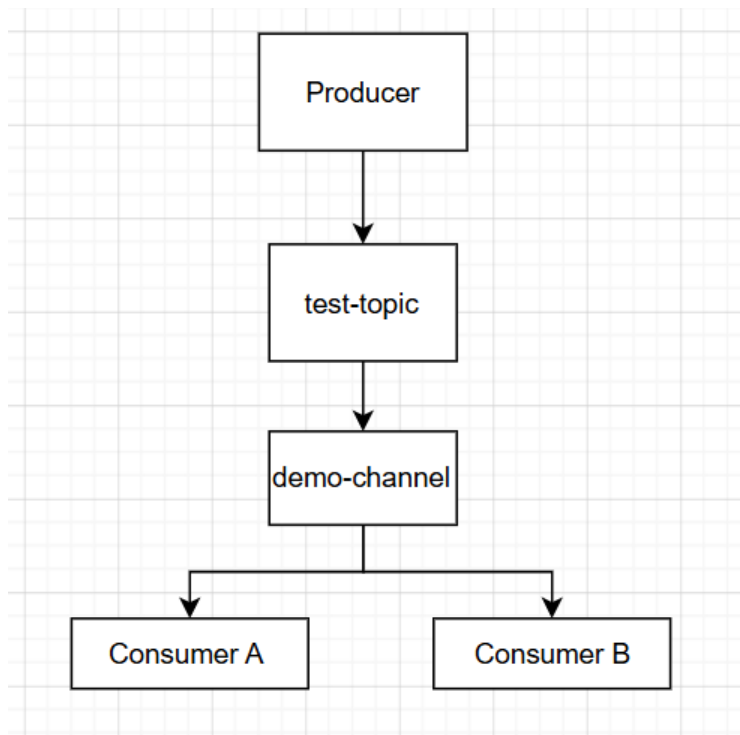
3.4 Thực nghiệm cân bằng tải (Load Balancing)

Nhóm xây dựng mô hình thực nghiệm gồm:

- 01 Producer
- 01 Topic (test-topic)
- 01 Channel (demo-channel)
- 02 Consumer

Hai Consumer cùng đăng ký vào một Channel để kiểm tra cơ chế phân phối thông điệp và cân bằng tải của NSQ.

Mô hình thử nghiệm:



Hình 4. Mô hình thực nghiệm cân bằng tải

Nhóm gửi 100 thông điệp vào Topic test-topic.

Kết quả nhận thông điệp:

- Consumer A nhận một phần các thông điệp được gửi đến hệ thống.
- Consumer B nhận phần thông điệp còn lại.

Client Connections

Client Host	User-Agent	Attributes	NSQd Host	In-Flight	Ready Count	Finished	Requeued	Messages	Uptime
codespaces-82ba45:45706	nsq_to_file/1.3.0 go-nsq/1.1.0		codespaces-82ba45:4151	0	200	49	0	49	2m33s
codespaces-82ba45:45704	nsq_to_file/1.3.0 go-nsq/1.1.0		codespaces-82ba45:4151	0	200	51	0	51	2m41s

Hình 5. Kết quả phân phối thông điệp giữa Consumer A và Consumer B.

Nhận xét:

Kết quả cho thấy các thông điệp được phân phối tự động giữa hai Consumer trong cùng một Channel. Số lượng thông điệp nhận được ở mỗi Consumer có thể không hoàn toàn bằng nhau nhưng đều được xử lý thành công.

Điều này chứng minh cơ chế cân bằng tải (Load Balancing) của NSQ hoạt động hiệu quả và giúp tối ưu tài nguyên xử lý.

3.5 Thực nghiệm khả năng chịu lỗi (Fault Tolerance)

Kịch bản thử nghiệm: Sau khi hệ thống hoạt động ổn định:

- Dừng Consumer A.
- Tiếp tục gửi thêm thông điệp mới.
- Quan sát khả năng xử lý của hệ thống.

Nhóm gửi tiếp 20 thông điệp:

Kết quả:

Client Connections

Client Host	User-Agent	Attributes	NSQd Host	In-Flight	Ready Count	Finished	Requeued	Messages	Uptime
codespaces-82ba45:45706	nsq_to_file/1.3.0 go-nsq/1.1.0		codespaces-82ba45:4151	0	200	69	0	69	5m5s

Hình 6. Consumer B nhận toàn bộ thông điệp

Toàn bộ thông điệp từ 101 đến 120 được Consumer B xử lý thành công. Consumer A không còn hoạt động nhưng hệ thống vẫn tiếp tục vận hành bình thường.

Nhận xét:

- Kết quả cho thấy NSQ có khả năng chịu lỗi tốt.
- Khi một Consumer gặp sự cố hoặc ngừng hoạt động, các Consumer còn lại vẫn tiếp tục nhận và xử lý thông điệp mà không làm gián đoạn hệ thống.
- Đây là một đặc điểm quan trọng của hệ thống phân tán hiện đại.

3.6 Đánh giá kết quả thực nghiệm

Qua quá trình cài đặt và thử nghiệm, nhóm nhận thấy:

- NSQ có kiến trúc đơn giản, dễ triển khai.
- Hệ thống hỗ trợ tốt cơ chế Publish/Subscribe.
- Thông điệp được phân phối tự động giữa các Consumer.
- Hệ thống hoạt động ổn định khi xảy ra lỗi Consumer.
- NSQ đáp ứng tốt các đặc tính của hệ thống phân tán gồm:
 - Khả năng mở rộng (Scalability)
 - Cân bằng tải (Load Balancing)
 - Khả năng chịu lỗi (Fault Tolerance)
 - Xử lý bất đồng bộ (Asynchronous Processing)

Kết quả thực nghiệm cho thấy NSQ là một nền tảng truyền thông điệp phân tán hiệu quả và phù hợp với các hệ thống yêu cầu xử lý dữ liệu thời gian thực.

4. PHÁT TRIỂN TÍNH NĂNG MỚI

Sau khi nghiên cứu kiến trúc và thực nghiệm hệ thống NSQ, nhóm tiến hành mở rộng dự án bằng cách phát triển thêm hai tính năng mới liên quan đến khả năng giám sát và quản trị hệ thống phân tán.

Các tính năng được triển khai trực tiếp trên giao diện quản trị NSQAdmin nhằm hỗ trợ người quản trị theo dõi trạng thái hoạt động của hệ thống một cách trực quan hơn.

4.1. Consumer Load Balancing Monitor

4.1.1. Mục đích

Trong hệ thống phân tán, nhiều Consumer có thể cùng tham gia xử lý thông điệp trong một Channel. Khi số lượng thông điệp tăng lên, hệ thống cần phân phối công việc giữa các Consumer để tránh tình trạng một Consumer bị quá tải trong khi các Consumer khác xử lý ít hoặc không xử lý.

NSQ đã hỗ trợ cơ chế phân phối thông điệp giữa các Consumer trong cùng một Channel. Tuy nhiên, phiên bản NSQAdmin gốc chưa cung cấp công cụ trực quan để theo dõi mức tải xử lý của từng Consumer.

Do đó, nhóm phát triển tính năng Consumer Load Balancing Monitor nhằm hỗ trợ người quản trị theo dõi mức tải của Consumer dựa trên số lượng thông điệp đã xử lý, từ đó đánh giá hiệu quả cân bằng tải trong hệ thống.

4.1.2. Thiết kế

Nhóm chỉnh sửa giao diện Channel Details trong NSQAdmin.

Tính năng mới bổ sung thêm cột Load Level trong bảng Client Connections để hiển thị mức tải của từng Consumer.

Mức tải được xác định dựa trên số lượng message mà Consumer đã xử lý.

Điều kiện	Load Level
Số message xử lý ≥ 100	HIGH LOAD
Số message xử lý từ 20 đến 99	MEDIUM LOAD
Số message xử lý < 20	LOW LOAD

Mỗi mức tải được hiển thị bằng màu sắc khác nhau nhằm giúp người quản trị dễ dàng nhận biết tình trạng hoạt động của Consumer.

4.1.3. Cài đặt

File được chỉnh sửa:

- nsqadmin/static/js/views/channel.hbs
- nsqadmin/static/js/lib/handlebars_helpers.js

Trong file channel.hbs, nhóm bổ sung cột Load Level vào bảng Client Connections để hiển thị mức tải của từng Consumer.

Trong file handlebars_helpers.js, nhóm xây dựng các helper hỗ trợ xác định mức tải và màu hiển thị tương ứng:

- consumer_load_level: xác định mức tải dựa trên số lượng message đã xử lý.
- consumer_load_class: xác định màu hiển thị tương ứng với từng mức tải.

Mức tải được hiển thị dưới dạng nhãn trực quan:

Messages Processed	Load Level
150	HIGH LOAD
35	MEDIUM LOAD
5	LOW LOAD

Thông tin được cập nhật trực tiếp trên giao diện NSQAdmin và không làm thay đổi cơ chế xử lý thông điệp gốc của NSQ.

4.1.4. Kết quả

Flight	Ready Count	Finished	Requeued	Messages	Load Level	Health Status	Uptime
	200	0	0	0	LOW LOAD	HEALTHY	22m7s
	200	149	0	149	HIGH LOAD	HEALTHY	31m37s

Hình 7. Giao diện Consumer Load Balancing Monitor

Kết quả cho thấy giao diện NSQAdmin đã hiển thị thêm cột Load Level trong bảng Client Connections.

Trong quá trình thực nghiệm, khi Consumer xử lý 150 message, hệ thống tự động đánh giá mức tải là HIGH LOAD và hiển thị trên giao diện quản trị.

Tính năng này giúp người quản trị nhanh chóng xác định Consumer nào đang xử lý nhiều thông điệp hơn các Consumer còn lại, từ đó hỗ trợ đánh giá hiệu quả phân phối tải trong hệ thống.

4.1.5. Liên hệ với hệ phân tán

Trong hệ thống phân tán, Load Balancing là một đặc tính quan trọng nhằm phân phối công việc giữa nhiều nút xử lý để tối ưu hiệu năng và tránh quá tải cục bộ.

Đối với NSQ, các Consumer đóng vai trò là các nút xử lý thông điệp. Khi nhiều Consumer cùng đăng ký vào một Channel, NSQ sẽ tự động phân phối thông điệp giữa các Consumer nhằm chia sẻ tải xử lý.

Tính năng Consumer Load Balancing Monitor không thay đổi thuật toán cân bằng tải của NSQ mà bổ sung khả năng giám sát mức tải của từng Consumer. Thông qua đó, quản trị viên có thể đánh giá hiệu quả phân phối thông điệp và khả năng mở rộng của hệ thống.

Tính năng này liên quan trực tiếp đến các kiến thức đã học trong môn Ứng dụng phân tán:

- Load Balancing.
- Scalability.
- Distributed Processing.

4.1.6. Đánh giá

Ưu điểm:

- Hỗ trợ theo dõi mức tải xử lý của từng Consumer.
- Giúp đánh giá hiệu quả cân bằng tải trong hệ thống NSQ.
- Giao diện trực quan, dễ quan sát.
- Không ảnh hưởng đến cơ chế xử lý thông điệp gốc của NSQ.
- Có thể mở rộng thành cơ chế cảnh báo Consumer quá tải trong tương lai.

Hạn chế:

- Mức tải hiện được đánh giá dựa trên số lượng message đã xử lý, chưa phản ánh chính xác mức sử dụng CPU hoặc bộ nhớ của Consumer.
- Chưa hỗ trợ cơ chế cảnh báo tự động khi Consumer bị quá tải.

Nhìn chung, tính năng Consumer Load Balancing Monitor đã đáp ứng tốt mục tiêu hỗ trợ giám sát hoạt động cân bằng tải của hệ thống phân tán và cung cấp thêm thông tin hữu ích cho người quản trị trong quá trình vận hành hệ thống.

4.2. Distributed Node Health Monitoring

4.2.1. Mục đích

Trong hệ thống phân tán, các node có thể chạy trên nhiều máy hoặc nhiều tiến trình khác nhau. Việc theo dõi trạng thái hoạt động của các node là rất quan trọng để phát hiện sớm sự cố, đảm bảo tính sẵn sàng và hỗ trợ vận hành hệ thống ổn định.

Trong NSQ, NSQD là thành phần tiếp nhận, lưu trữ và phân phối thông điệp. Nếu một node NSQD gặp sự cố hoặc mất kết nối, khả năng xử lý thông điệp của hệ thống có thể bị ảnh hưởng.

Tuy nhiên, phiên bản NSQAdmin gốc chưa cung cấp cơ chế hiển thị trực quan trạng thái hoạt động của các node NSQD. Người quản trị phải quan sát nhiều thông tin khác nhau để đánh giá tình trạng của node.

Do đó, nhóm phát triển tính năng Distributed Node Health Monitoring nhằm hỗ trợ giám sát trạng thái hoạt động của các node NSQD trên giao diện NSQAdmin.

4.2.2. Thiết kế

Nhóm chỉnh sửa giao diện danh sách node trong NSQAdmin.

Tính năng mới bổ sung cột Node Health trên trang Nodes để hiển thị trạng thái hoạt động của từng node NSQD.

Trạng thái của node được xác định dựa trên các thông tin mà NSQAdmin thu thập được từ NSQD và NSQLookupd.

Điều kiện	Node Health
Node có Topic hoặc có kết nối Lookupd	UP
Node không có Topic và không có kết nối Lookupd	WARNING

Bảng 4.3. Trạng thái Node Health

Mỗi trạng thái được hiển thị bằng màu sắc khác nhau nhằm giúp người quản trị dễ dàng nhận biết tình trạng hoạt động của node.

4.2.3. Cài đặt

File được chỉnh sửa:

- nsqadmin/static/js/views/nodes.hbs
- nsqadmin/static/js/lib/handlebars_helpers.js

Trong file `nodes.hbs`, nhóm bổ sung cột Node Health vào bảng danh sách NSQD Nodes.

Trong file `handlebars_helpers.js`, nhóm xây dựng các helper:

- `node_health_status`: xác định trạng thái hoạt động của node.
- `node_health_class`: xác định màu hiển thị tương ứng với trạng thái node.

Thông tin được hiển thị trực tiếp trên giao diện NSQAdmin dưới dạng nhãn trực quan.

4.2.4. Kết quả

Region	Zone	Lookupd Conns.	Node Health	Topics
		1	UP	1 test-topic

Hình 8. Giao diện Distributed Node Health Monitoring

Kết quả cho thấy giao diện NSQAdmin đã hiển thị thêm cột Node Health trên trang Nodes.

Trong quá trình thực nghiệm, node NSQD đang hoạt động bình thường được hiển thị trạng thái UP.

Người quản trị có thể quan sát trực tiếp trạng thái hoạt động của các node mà không cần truy cập vào từng node riêng lẻ.

Tính năng này giúp việc giám sát hệ thống phân tán trở nên trực quan và thuận tiện hơn.

4.2.5. Kiểm thử

STT	Trường hợp kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
1	Node NSQD đang hoạt động và có Topic	Node Health = UP	Hiển thị UP	PASS
2	Node NSQD có kết nối Lookupd	Node Health = UP	Hiển thị UP	PASS

3	Node không có Topic và không có kết nối Lookupd	Node Health = WARNING	Hiển thị WARNING	PASS
4	Truy cập trang Nodes trên NSQAdmin	Hiển thị cột Node Health	Cột Node Health được hiển thị	PASS

4.2.6. Liên hệ với hệ phân tán

Trong hệ thống phân tán, các thành phần xử lý thường được triển khai trên nhiều node khác nhau. Khi một node gặp lỗi hoặc mất kết nối, khả năng hoạt động của toàn hệ thống có thể bị ảnh hưởng.

Việc giám sát trạng thái hoạt động của các node giúp quản trị viên phát hiện sớm các dấu hiệu bất thường, từ đó đưa ra biện pháp xử lý phù hợp nhằm duy trì tính ổn định của hệ thống.

Tính năng Distributed Node Health Monitoring không thay đổi cơ chế hoạt động của NSQ mà bổ sung khả năng giám sát trạng thái node trên giao diện NSQAdmin.

Thông qua tính năng này, người quản trị có thể đánh giá tính sẵn sàng và khả năng chịu lỗi của hệ thống phân tán.

Tính năng liên quan trực tiếp đến các kiến thức đã học trong môn Ứng dụng phân tán:

- Fault Detection.
- Fault Tolerance.
- Availability.
- Reliability.

4.2.7. Đánh giá

Ưu điểm:

- Hỗ trợ giám sát trạng thái hoạt động của các node NSQD.
- Giúp phát hiện sớm các node có dấu hiệu bất thường.
- Hỗ trợ đánh giá tính sẵn sàng của hệ thống phân tán.
- Giao diện trực quan, dễ sử dụng.
- Không ảnh hưởng đến hiệu năng hoạt động của hệ thống.

Hạn chế:

- Trạng thái hiện tại mới được xác định dựa trên thông tin kết nối và dữ liệu node.
- Chưa hỗ trợ cơ chế cảnh báo tự động khi node gặp sự cố.
- Chưa tích hợp các chỉ số chuyên sâu như CPU, RAM hoặc độ trễ mạng.

Nhìn chung, tính năng Distributed Node Health Monitoring đã đáp ứng tốt mục tiêu hỗ trợ giám sát hoạt động của các node trong hệ thống phân tán và cung cấp thêm thông tin hữu ích cho người quản trị trong quá trình vận hành hệ thống.

4.3 Tổng kết các tính năng mở rộng

STT	Tính năng	Mục đích
1	Consumer Load Balancing Monitor	Theo dõi mức tải của từng Consumer dựa trên số lượng thông điệp đã xử lý. Tính năng hỗ trợ đánh giá hiệu quả cân bằng tải (Load Balancing), khả năng phân phối công việc giữa các Consumer và khả năng mở rộng (Scalability) của hệ thống phân tán.
2	Distributed Node Health Monitoring	Theo dõi trạng thái hoạt động của các node NSQD trong hệ thống. Tính năng hỗ trợ phát hiện node bất thường (Fault Detection), nâng cao khả năng chịu lỗi (Fault Tolerance), tính sẵn sàng (Availability) và hỗ trợ giám sát các thành phần trong môi trường phân tán.

Bảng 4.1. Tổng hợp tính năng mới

Kết quả thực nghiệm cho thấy cả hai tính năng đều hoạt động ổn định trên hệ thống NSQAdmin.

Đối với tính năng Consumer Load Balancing Monitor, hệ thống có khả năng đánh giá mức tải của từng Consumer thông qua số lượng thông điệp đã xử lý và hiển thị dưới dạng các mức LOW LOAD, MEDIUM LOAD và HIGH LOAD. Điều này giúp người quản trị dễ dàng theo dõi hiệu quả phân phối tải giữa các Consumer trong cùng một Channel.

Đối với tính năng Distributed Node Health Monitoring, hệ thống có khả năng hiển thị trạng thái hoạt động của các node NSQD thông qua chỉ số Node Health. Nhờ đó người quản trị có thể nhanh chóng phát hiện các node có dấu hiệu bất thường và đánh giá tình trạng hoạt động của hệ thống.

Hai tính năng được xây dựng dựa trên các kiến thức đã học trong môn Ứng dụng phân tán, đặc biệt là các khái niệm về Load Balancing, Scalability, Fault Detection, Fault Tolerance và Availability.

Mặc dù các tính năng mới không can thiệp trực tiếp vào thuật toán xử lý phân tán của NSQ, nhưng chúng đóng vai trò hỗ trợ giám sát và quản trị hệ thống, giúp người vận hành theo dõi các đặc tính phân tán một cách trực quan và hiệu quả hơn.

Thông qua việc triển khai hai tính năng mở rộng này, nhóm đã vận dụng được các kiến thức lý thuyết vào một hệ thống phân tán thực tế, đồng thời nâng cao khả năng giám sát và quản trị của NSQAdmin so với phiên bản ban đầu.

Trong tương lai, hệ thống có thể tiếp tục được mở rộng theo các hướng như:

- Consumer Failure Detection.
- Multi-Node Monitoring.
- Dead Letter Queue Monitoring.
- Distributed Performance Dashboard.

Các hướng phát triển trên sẽ giúp tăng cường khả năng giám sát, phân tích và vận hành hệ thống phân tán trong các môi trường triển khai thực tế có quy mô lớn hơn.

Nhìn chung, hai tính năng mới đã được triển khai thành công, đáp ứng mục tiêu đề ra và góp phần nâng cao khả năng quản trị, giám sát cũng như đánh giá hoạt động của hệ thống truyền thông điệp phân tán NSQ.

5. ĐÓNG GÓP CỦA THÀNH VIÊN

5.1 Phân công công việc

Trong quá trình thực hiện đề tài, các thành viên trong nhóm đã phối hợp nghiên cứu, cài đặt, thực nghiệm và phát triển tính năng mới cho hệ thống NSQ. Việc phân công công việc được thực hiện dựa trên khả năng và thời gian tham gia của từng thành viên.

Thành viên	Công việc thực hiện
Nguyễn Thị Thùy Linh	Tìm hiểu kiến trúc hệ thống NSQ và các thành phần NSQD, NSQLookupd, NSQAdmin; thiết lập môi trường GitHub Codespaces; cài đặt và cấu hình hệ thống; thực hiện thực nghiệm Load Balancing và Fault Tolerance; phát triển tính năng Consumer Load Balancing Monitor; tổng hợp kết quả thực nghiệm; tham gia xây dựng báo cáo và slide thuyết trình.
Nguyễn Anh Quân	Nghiên cứu cơ chế hoạt động của Producer, Consumer, Topic và Channel; hỗ trợ triển khai và kiểm thử hệ thống; thực hiện thực nghiệm truyền nhận thông điệp; phát triển tính năng Distributed Node Health Monitoring; hỗ trợ chỉnh sửa mã nguồn và giao diện NSQAdmin; tham gia xây dựng báo cáo và slide thuyết trình.

5.2 Kết quả làm việc nhóm

Nhóm đã hoàn thành đầy đủ các yêu cầu của bài tập giữa kỳ bao gồm:

- Tìm hiểu và nghiên cứu dự án NSQ.
- Cài đặt thành công hệ thống NSQ trên GitHub Codespaces.
- Triển khai và vận hành thành công các thành phần NSQLookupd, NSQD và NSQAdmin.
- Thực nghiệm cơ chế Publish/Subscribe.
- Thực nghiệm cân bằng tải (Load Balancing) giữa nhiều Consumer.
- Thực nghiệm khả năng chịu lỗi (Fault Tolerance) của hệ thống.
- Phát triển thành công hai tính năng mới:
 - Consumer Load Balancing Monitor.
 - Distributed Node Health Monitoring.
- Quản lý mã nguồn dự án trên GitHub.
- Hoàn thiện báo cáo và slide trình bày.

Thông qua quá trình thực hiện đề tài, các thành viên đã hiểu rõ hơn về kiến trúc hệ thống phân tán, cơ chế truyền thông điệp bất đồng bộ, khả năng mở rộng, cân bằng tải và khả năng chịu lỗi của các nền tảng Distributed Messaging hiện đại.

Bên cạnh đó, nhóm cũng có cơ hội tiếp cận với việc nghiên cứu mã nguồn mở, tùy chỉnh giao diện quản trị NSQAdmin và phát triển các chức năng hỗ trợ giám sát hệ thống phân tán dựa trên các kiến thức đã học trong môn Ứng dụng phân tán.

Kết quả đạt được cho thấy nhóm đã hoàn thành đầy đủ các mục tiêu đề ra, đồng thời vận dụng được các kiến thức lý thuyết vào một hệ thống phân tán thực tế.

KẾT LUẬN

Trong đề tài này, nhóm đã nghiên cứu, cài đặt và thực nghiệm thành công hệ thống truyền thông điệp phân tán NSQ. Kết quả thực nghiệm cho thấy NSQ đáp ứng tốt các yêu cầu của một hệ thống phân tán hiện đại như xử lý bất đồng bộ, cân bằng tải giữa nhiều Consumer và khả năng chịu lỗi khi xảy ra sự cố trong quá trình vận hành.

Bên cạnh việc tìm hiểu kiến trúc và cơ chế hoạt động của NSQ, nhóm đã triển khai thành công hai tính năng mở rộng gồm Priority-Based Topic Classification and Monitoring và Consumer Health Monitoring trên giao diện NSQAdmin. Các tính năng này góp phần nâng cao khả năng giám sát và quản trị hệ thống, đồng thời thể hiện khả năng mở rộng và tùy biến của dự án mã nguồn mở.

Thông qua quá trình thực hiện đề tài, nhóm đã củng cố kiến thức về hệ phân tán, cơ chế Publish/Subscribe, Service Discovery, Load Balancing, Fault Tolerance và các phương pháp giám sát hệ thống phân tán. Đây là những kiến thức quan trọng có thể được áp dụng trong việc xây dựng các hệ thống Microservices, hệ thống xử lý dữ liệu thời gian thực và các ứng dụng quy mô lớn trong thực tế.

Trong tương lai, các tính năng đã phát triển có thể được mở rộng theo hướng xây dựng cơ chế ưu tiên xử lý thông điệp, cảnh báo khi Consumer hoạt động bất thường và hỗ trợ giám sát hệ thống theo thời gian thực nhằm nâng cao hiệu quả vận hành của nền tảng NSQ.